
Virtual Memory

Page Tables and Memory Management in xv6

Contents

- Overview
 - Virtual memory
 - Paging
- Page table
 - User page table creation
 - Kernel page table creation and initialization
 - Page table switching

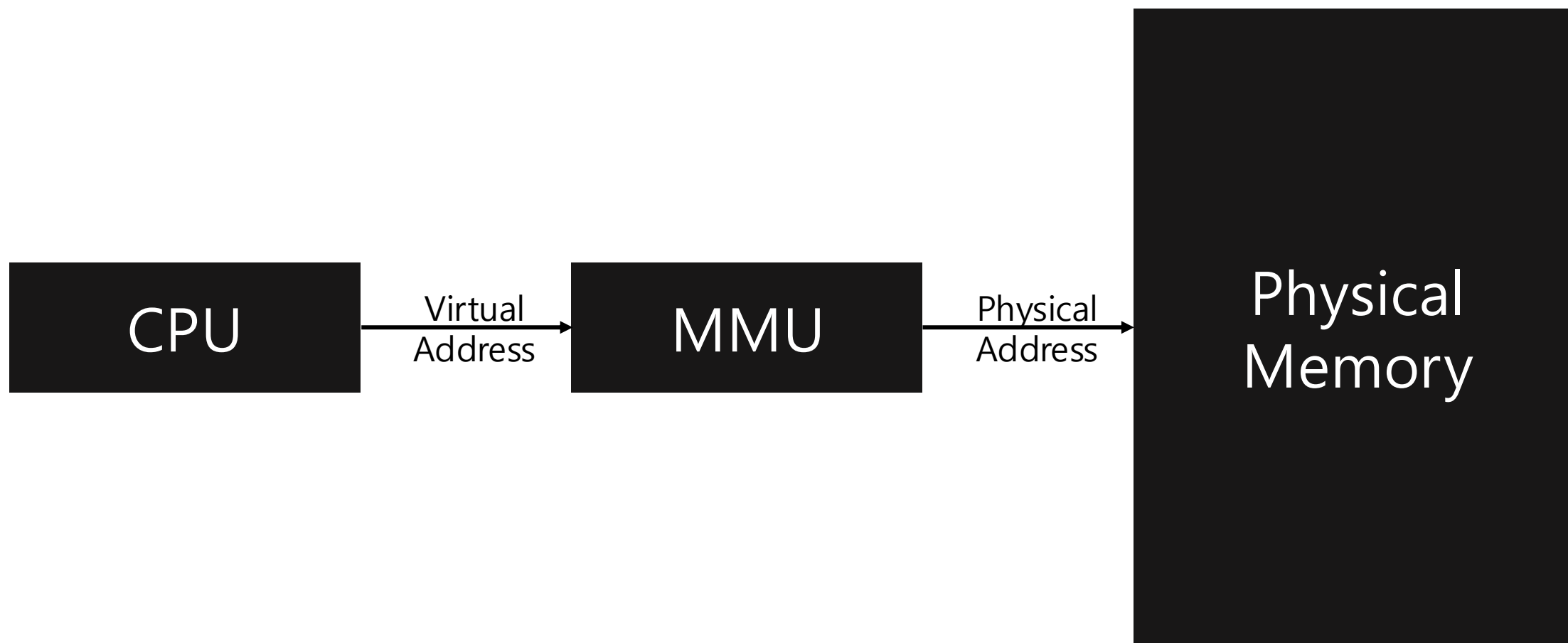
Virtual Memory

Overview

What is Virtual Memory?

- If a process can address 8GB of memory, and 1,000 processes are running... do we need 8TB of physical RAM?
 - **No**, OS provides Virtual Memory.
- **Virtual Memory**
 - An OS technique that allows programs to use a memory space larger than the actual physical RAM afforded.
 - It provides the **illusion** of a large, continuous block of memory to each program.
 - Each process operates in its own isolated **logical (virtual) address space**.
 - The CPU generates virtual addresses, which the OS dynamically maps to physical RAM (or disk) behind the scenes.

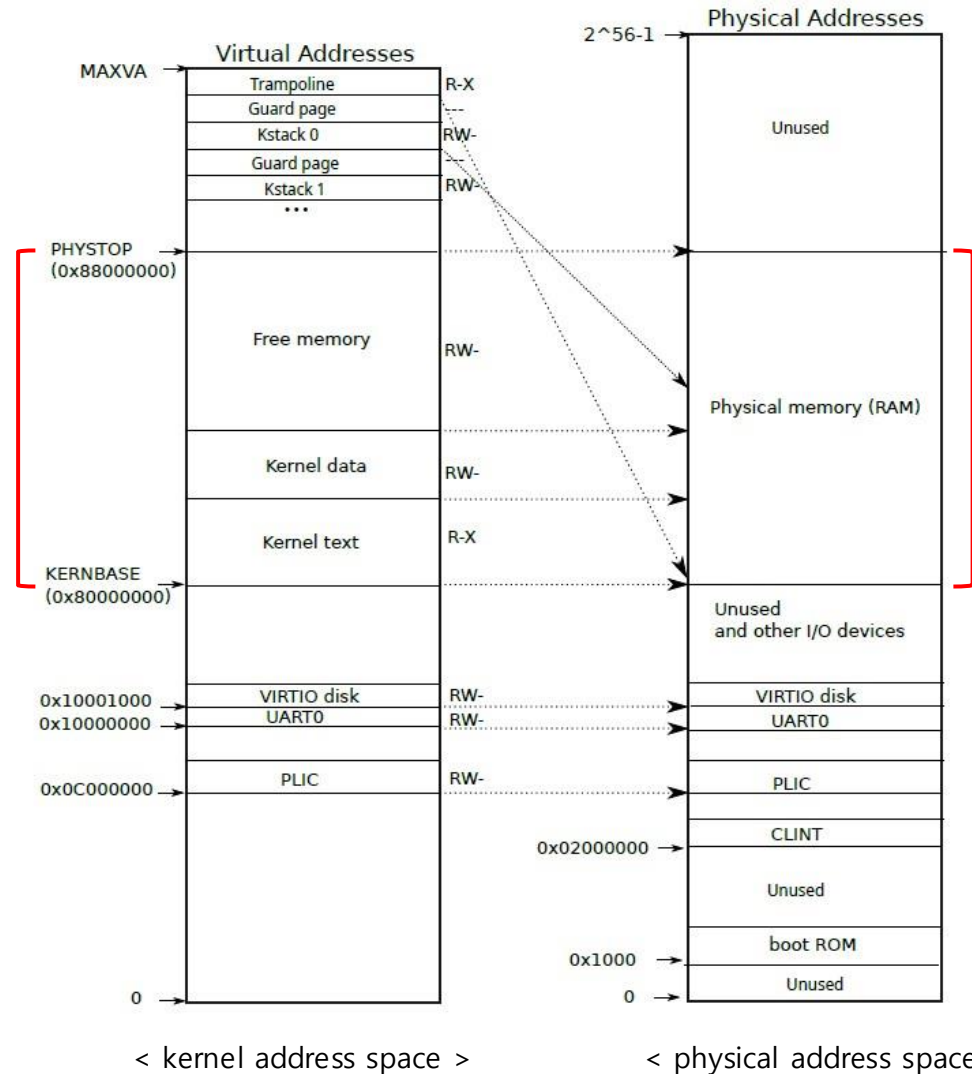
Virtual Memory Translation



Paging

- **Paging** is a memory management scheme used by operating systems where a process's logical address space is divided into fixed-size blocks called **pages**, and the physical memory is divided into blocks of the same size called **frames**.
- OS uses **page table** to map **virtual page** numbers to **physical memory frame** numbers.
 - The page table consulted to obtain the frame number that corresponds to the given page number.
 - With frame number and page number, MMU can calculate physical address.

Page Allocation



Memory pages come from the free memory region located between KERNBASE and PHYSTOP.

This region is mapped 1:1 to virtual addresses in the kernel page table.

Physical Memory Management in xv6

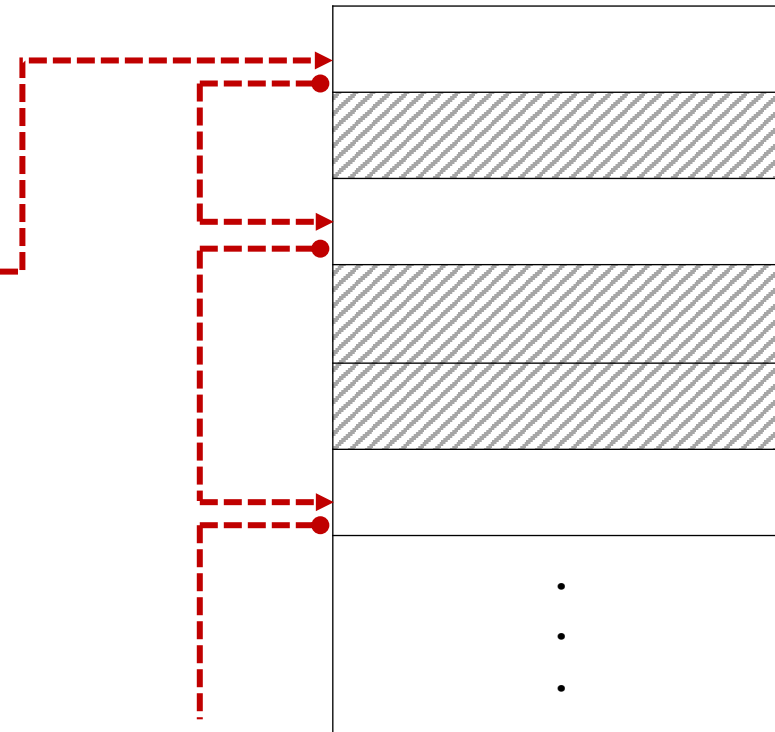
- Xv6 manages **free list** of physical pages that are available for allocation
 - Free pages are linked as a single linked-list

```
$ vim kernel/kalloc.c
```

```
21 struct {  
22     struct spinlock lock;  
23     struct run *freelist;  
24 } kmem;
```

```
$ vim kernel/kalloc.c
```

```
17 struct run {  
18     struct run *next;  
19 };
```



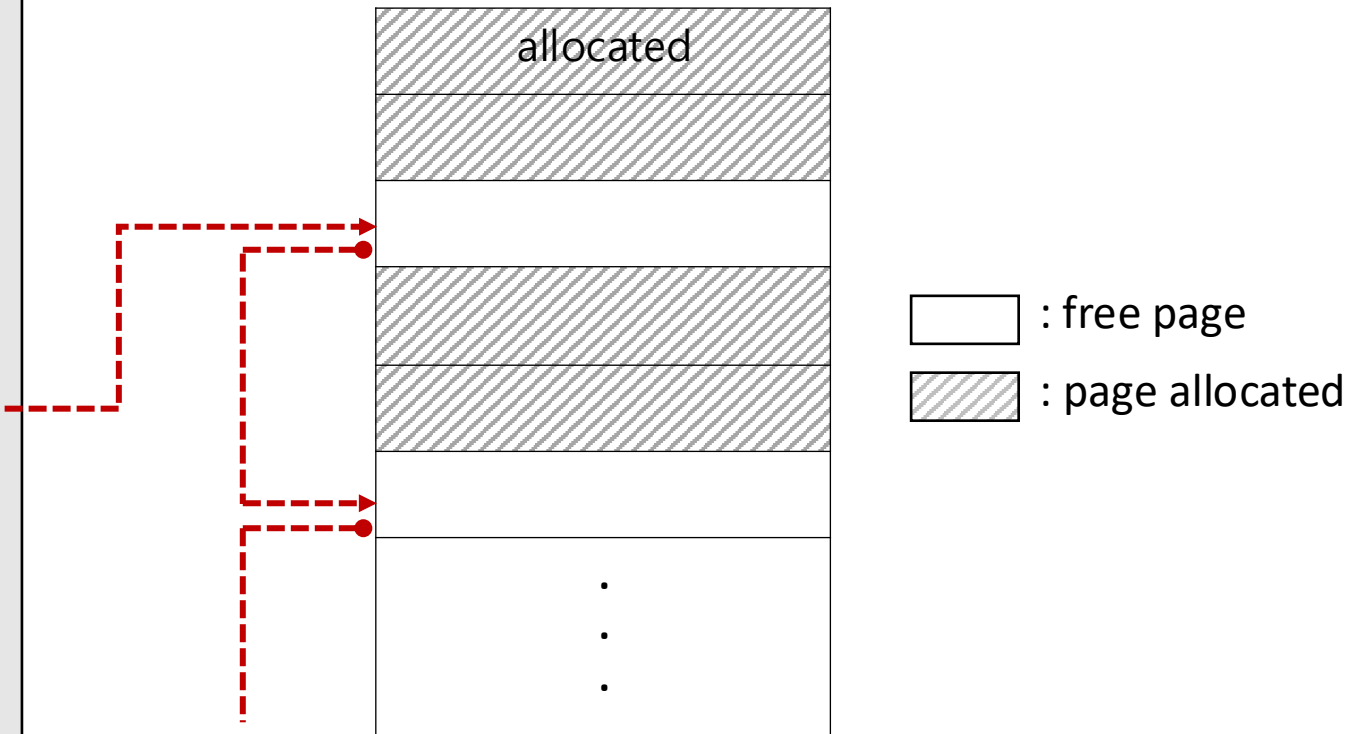
□ : free page
▨ : page allocated

Physical Memory Management in xv6

- `kalloc()` removes and returns the first element in the free list.

```
$ vim kernel/kalloc.c
```

```
68 void *  
69 kalloc(void)  
70 {  
71     struct run *r;  
72  
73     acquire(&kmem.lock);  
74     r = kmem.freelist;  
75     if(r)  
76         kmem.freelist = r->next;  
77     release(&kmem.lock);  
78  
79     if(r)  
80         memset((char*)r, 5, PGSIZE);  
81     return (void*)r;  
82 }
```

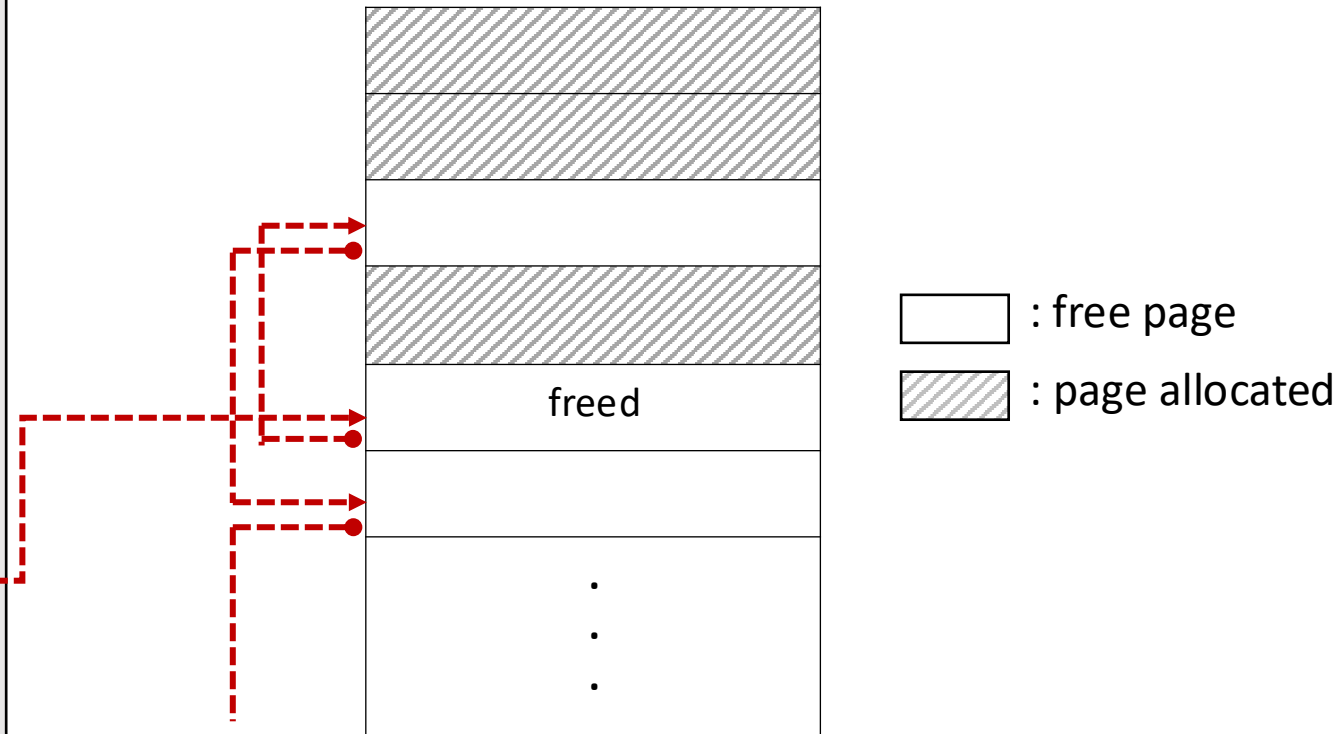


Physical Memory Management in xv6

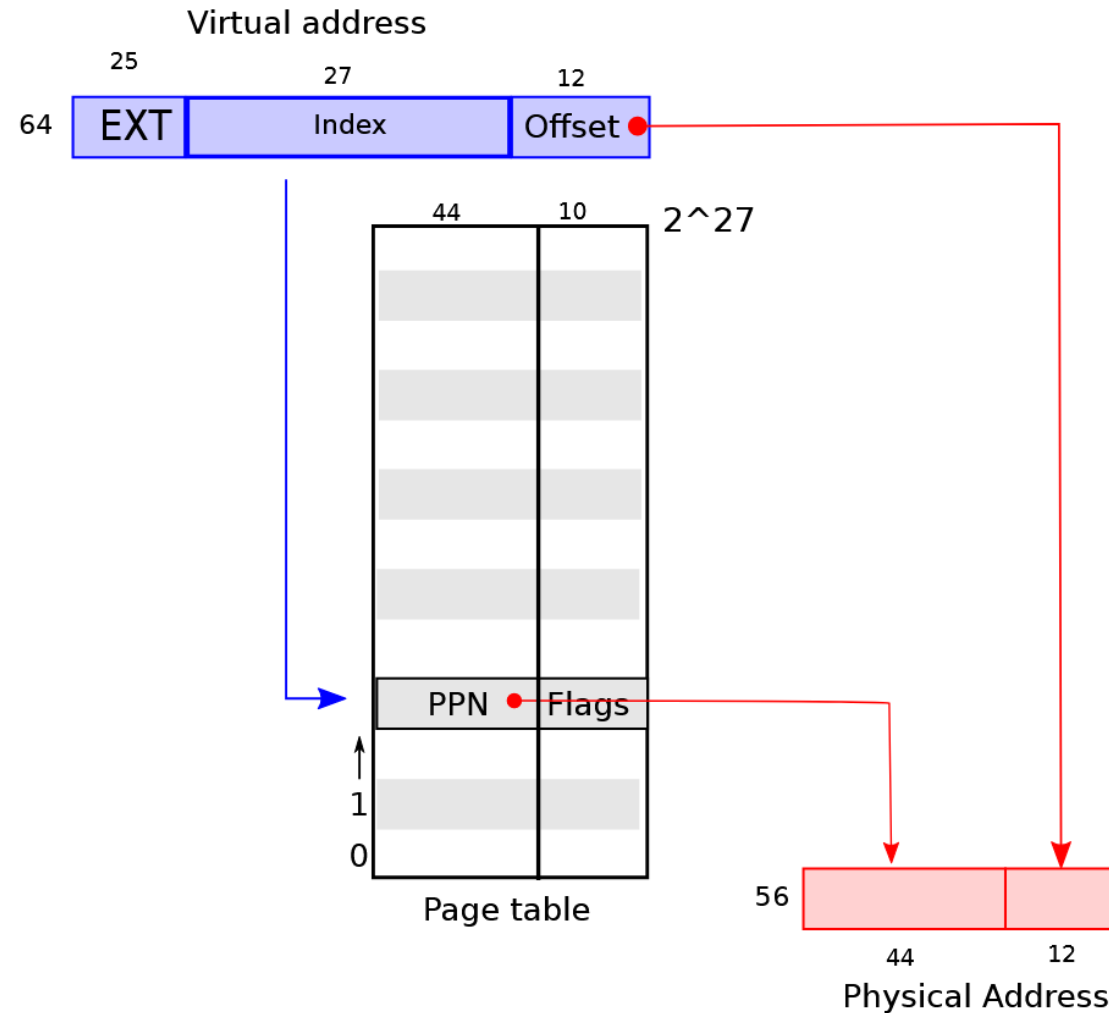
- kfree() prepends a page into the free list
 - So, this page is available for allocation again.

```
$ vim kernel/kalloc.c
```

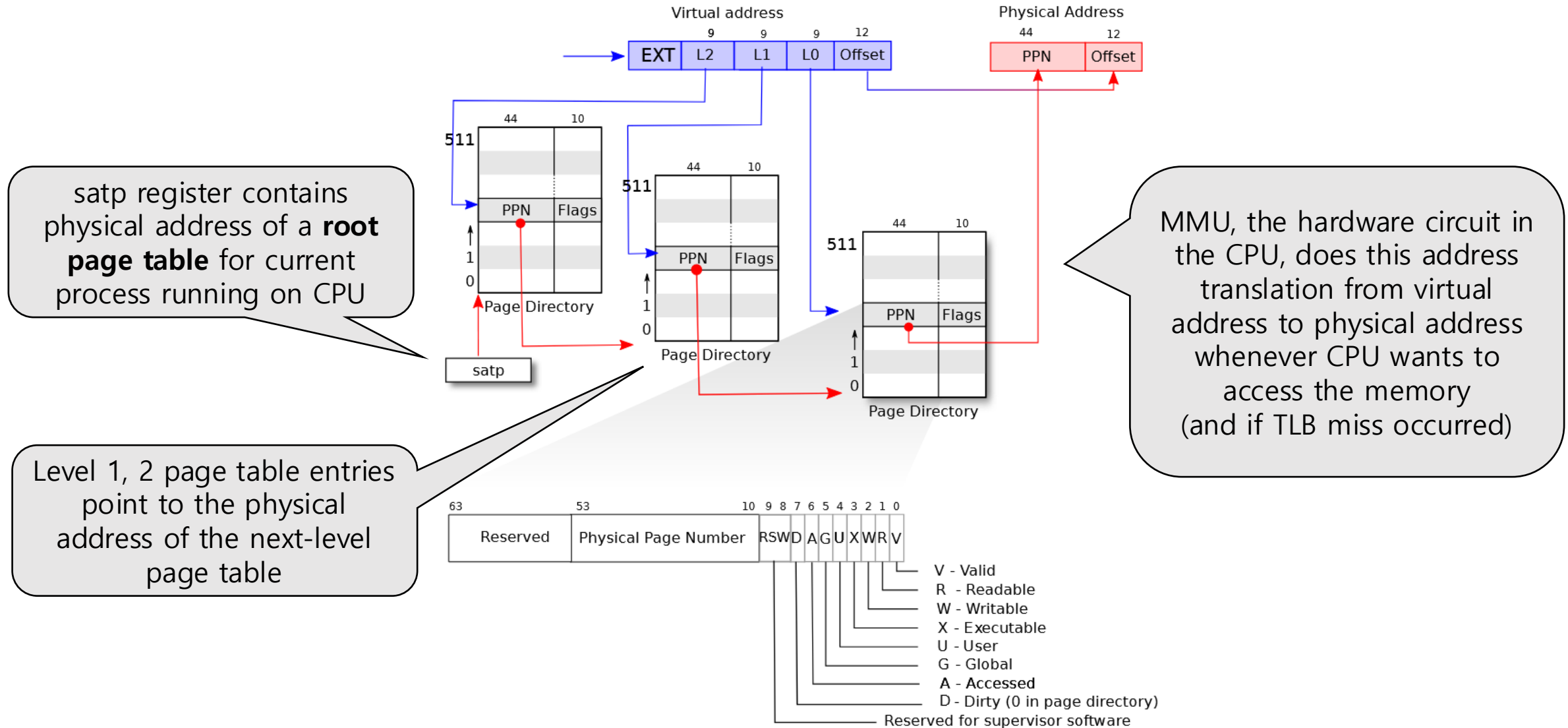
```
46 void *  
47 kfree(void *pa)  
48 {  
49     struct run *r;  
...  
57     r = (struct run*)pa;  
58  
59     acquire(&kmem.lock);  
60     r->next = kmem.freelist;  
61     kmem.freelist = r;  
62     release(&kmem.lock);  
63 }
```



Address Translation (simplified)



Address Translation



Virtual Memory

About Page Table

Page Table in xv6

- Page table for each process

```
$ vim kernel/proc.h

85  struct proc {
...
101     pagetable_t pagetable; // User page table
...
107 };
```

- Page table for kernel

```
$ vim kernel/vm.c

11  /*
12   * the kernel's page table.
13   */
14  pagetable_t kernel_pagetable;
...
```

Lookup Virtual Address in Page Table

```
$ vim kernel/vm.c

120  uint64
121  walkaddr(pagetable_t pagetable, uint64 va)
122  {
123      pte_t *pte;
124      uint64 pa;
125
126      if(va >= MAXVA)
127          return 0;
128
129      pte = walk(pagetable, va, 0);
130      if(pte == 0)
131          return 0;
132      if((*pte & PTE_V) == 0)
133          return 0;
134      if((*pte & PTE_U) == 0)
135          return 0;
136      pa = PTE2PA(*pte);
137      return pa;
138  }
```

- **walkaddr()** looks up virtual address and return the physical address.
 - Use **walk()** to get page table entry.
 - Verify and convert page table entry to the physical address.

Lookup Virtual Address in Page Table

```
$ vim kernel/vm.c

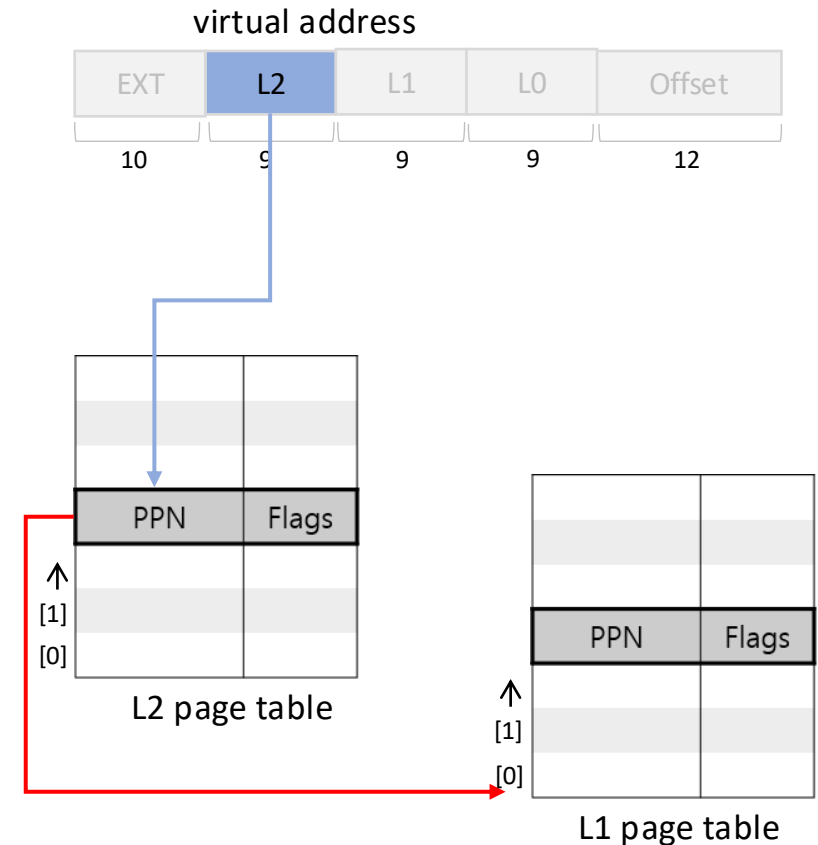
97  pte_t *
98  walk(pagetable_t pagetable, uint64 va, int alloc)
99  {
100     if(va >= MAXVA)
101         panic("walk");
102
103     for(int level = 2; level > 0; level--) {
104         pte_t *pte = &pagetable[PX(level, va)];
105         if(*pte & PTE_V) {
106             pagetable = (pagetable_t)PTE2PA(*pte);
107         } else {
108             if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
109                 return 0;
110             memset(pagetable, 0, PGSIZE);
111             *pte = PA2PTE(pagetable) | PTE_V;
112         }
113     }
114     return &pagetable[PX(0, va)];
115 }
```

- **walk()** searches certain virtual page's page table entry.
 - implements RISC-V paging hardware execution

Lookup Virtual Address in Page Table

```
$ vim kernel/vm.c
```

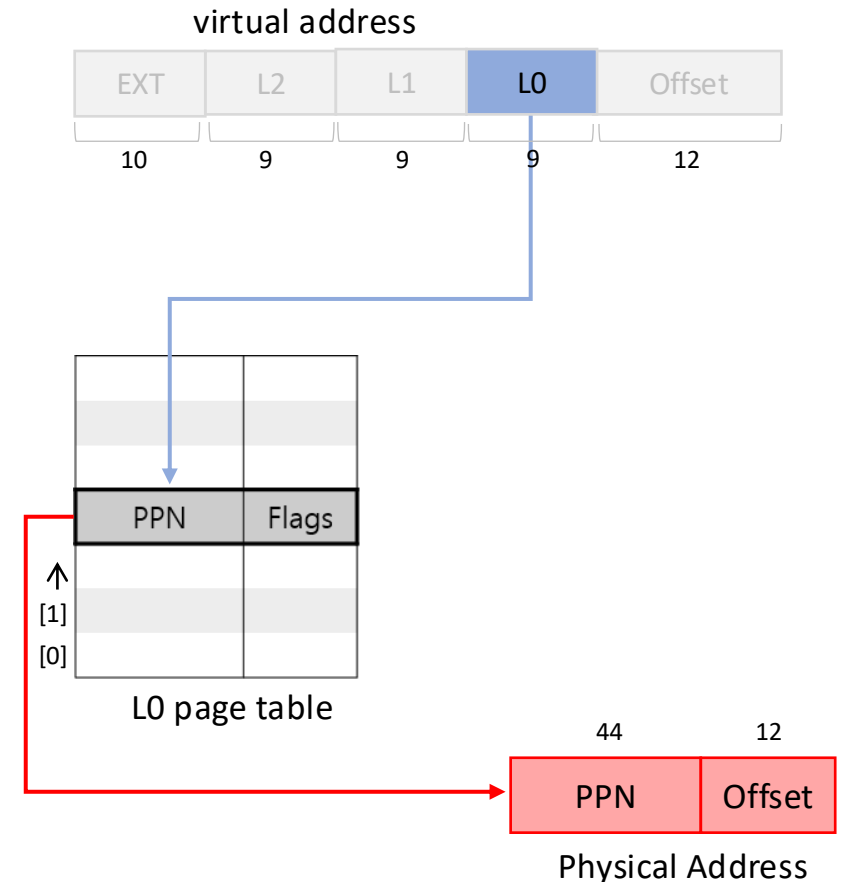
```
97 pte_t *  
98 walk(pagetable_t pagetable, uint64 va, int alloc)  
99 {  
100     if(va >= MAXVA)  
101         panic("walk");  
102  
103     for(int level = 2; level > 0; level--) {  
104         pte_t *pte = &pagetable[PX(level, va)];  
105         if(*pte & PTE_V) {  
106             pagetable = (pagetable_t)PTE2PA(*pte);  
107         } else {  
108             if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)  
109                 return 0;  
110             memset(pagetable, 0, PGSIZE);  
111             *pte = PA2PTE(pagetable) | PTE_V;  
112         }  
113     }  
114     return &pagetable[PX(0, va)];  
115 }
```



Lookup Virtual Address in Page Table

```
$ vim kernel/vm.c
```

```
97 pte_t *
98 walk(pagetable_t pagetable, uint64 va, int alloc)
99 {
100     if(va >= MAXVA)
101         panic("walk");
102
103     for(int level = 2; level > 0; level--) {
104         pte_t *pte = &pagetable[PX(level, va)];
105         if(*pte & PTE_V) {
106             pagetable = (pagetable_t)PTE2PA(*pte);
107         } else {
108             if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
109                 return 0;
110             memset(pagetable, 0, PGSIZE);
111             *pte = PA2PTE(pagetable) | PTE_V;
112         }
113     }
114     return &pagetable[PX(0, va)];
115 }
```



Create User Process's Page Table

```
$ vim kernel/proc.c

109 static struct proc*
110 allocproc(void)
111 {
...
124 found:
...
129 if((p->trapframe = (struct trapframe *)kalloc(...
...
134
135 // An empty user page table.
136 p->pagetable = proc_pagetable(p);
137 if(p->pagetable == 0){
138     freeproc(p);
139     release(&p->lock);
140     return 0;
141 }
...
```

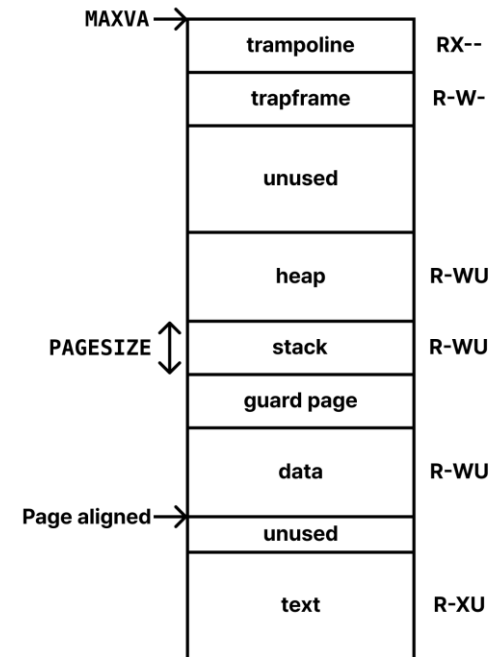
- During process creation, a new user page table is initialized to define the virtual memory layout for the process.
 - by `proc_pagetable()`

Create User Process's Page Table

```
$ vim kernel/proc.c

176 pagetable_t
177 proc_pagetable(struct proc *p)
178 {
179     pagetable_t pagetable;
180     ...
181     // An empty page table.
182     pagetable = uvmcreate();
183     ...
190     if(mappages(pagetable, TRAMPOLINE, PGSIZE,
191                 (uint64)trampoline, PTE_R | PTE_X) < 0){
192         release(&p->lock);
193         return 0;
194     }
195     ...
198     if(mappages(pagetable, TRAPFRAME, PGSIZE,
199                 (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
200         uvmunmap(pagetable, TRAMPOLINE, 1, 0);
201         uvmfree(pagetable, 0);
202         return 0;
203     }
204
205     return pagetable;
206 }
```

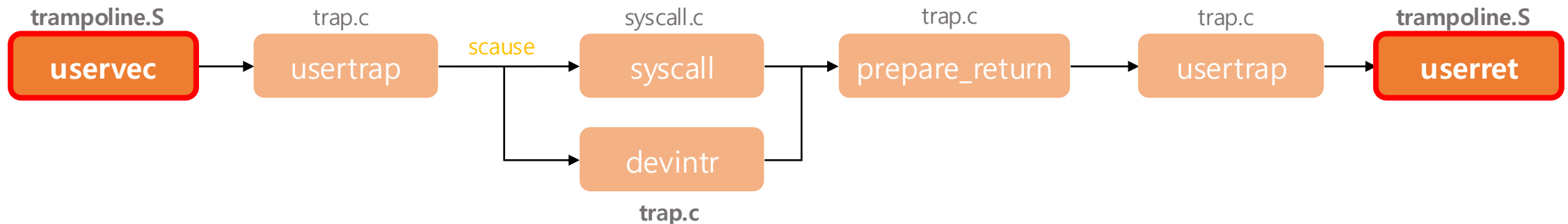
- **proc_pagetable()** creates and initializes process's page table.
 - Create an empty page table.
(L2 root page table only)
 - Map trampoline page
 - Map trapframe page (per process)



A process's user address space

Trampoline Page

- Contains core code(**uservec**, **userret**) for trap handling.
 - A single physical page is mapped to the same top VA (TRAMPOLINE) in all user page tables and the kernel page table.
- Ensures trap handling code remains accessible and executable even when the page table is switched.



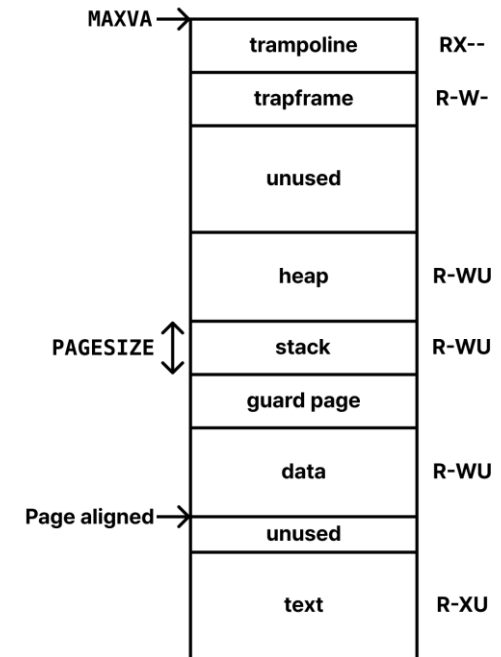
Create User Process's Page Table

```
$ vim kernel/proc.c
```

```
176 pagetable_t
177 proc_pagetable(struct proc *p)
178 {
179     pagetable_t pagetable;
180     ...
181     // An empty page table.
182     pagetable = uvmcreate();
183     ...
190     if(mappages(pagetable, TRAMPOLINE, PGSIZE,
191                 (uint64)trampoline, PTE_R | PTE_X) < 0){
192         release(&p->lock);
193         return 0;
194     }
195     ...
198     if(mappages(pagetable, TRAPFRAME, PGSIZE,
199                 (uint64)(p->trapframe), PTE_R | PTE_W) < 0){
200         uvmunmap(pagetable, TRAMPOLINE, 1, 0);
201         uvmfree(pagetable, 0);
202         return 0;
203     }
204     ...
205     return pagetable;
206 }
```

- **proc_pagetable()** creates and initializes process's page table.

- Create an empty page table.
(L2 root page table only)
- Map trampoline page
- Map trapframe page (per process)



A process's user address space

uvmcreate

```
$ vim kernel/vm.c
```

```
178 pagetable_t
179 uvmcreate()
180 {
181     pagetable_t pagetable;
182     pagetable = (pagetable_t) kalloc();
183     if(pagetable == 0)
184         return 0;
185     memset(pagetable, 0, PGSIZE);
186     return pagetable;
187 }
```

- uvmcreate() allocates a single empty page and initializes it as a user-level root page table.
 - And returns its address.

mappages

```
$ vim kernel/vm.c
```

```
145 int
146 mappages(paetable_t paetable, uint64 va, uint64 size, uint64 pa, int perm)
147 {
...
160     a = va;
161     last = va + size - PGSIZE;
162     for(;;){
163         if((pte = walk(paetable, a, 1)) == 0)
164             return -1;
165         if(*pte & PTE_V)
166             panic("mappages: remap");
167         *pte = PA2PTE(pa) | perm | PTE_V;
168         if(a == last)
169             break;
170         a += PGSIZE;
171         pa += PGSIZE;
172     }
173     return 0;
174 }
```

- **mappages()** divides the given virtual address range into page-sized units and maps each page to the corresponding physical address within the specified physical address range.
- This function establishes the relationship between virtual memory and physical memory by creating entries in the page table.

Create and Initialize Kernel's Page Table

```
$ vim kernel/vm.c
```

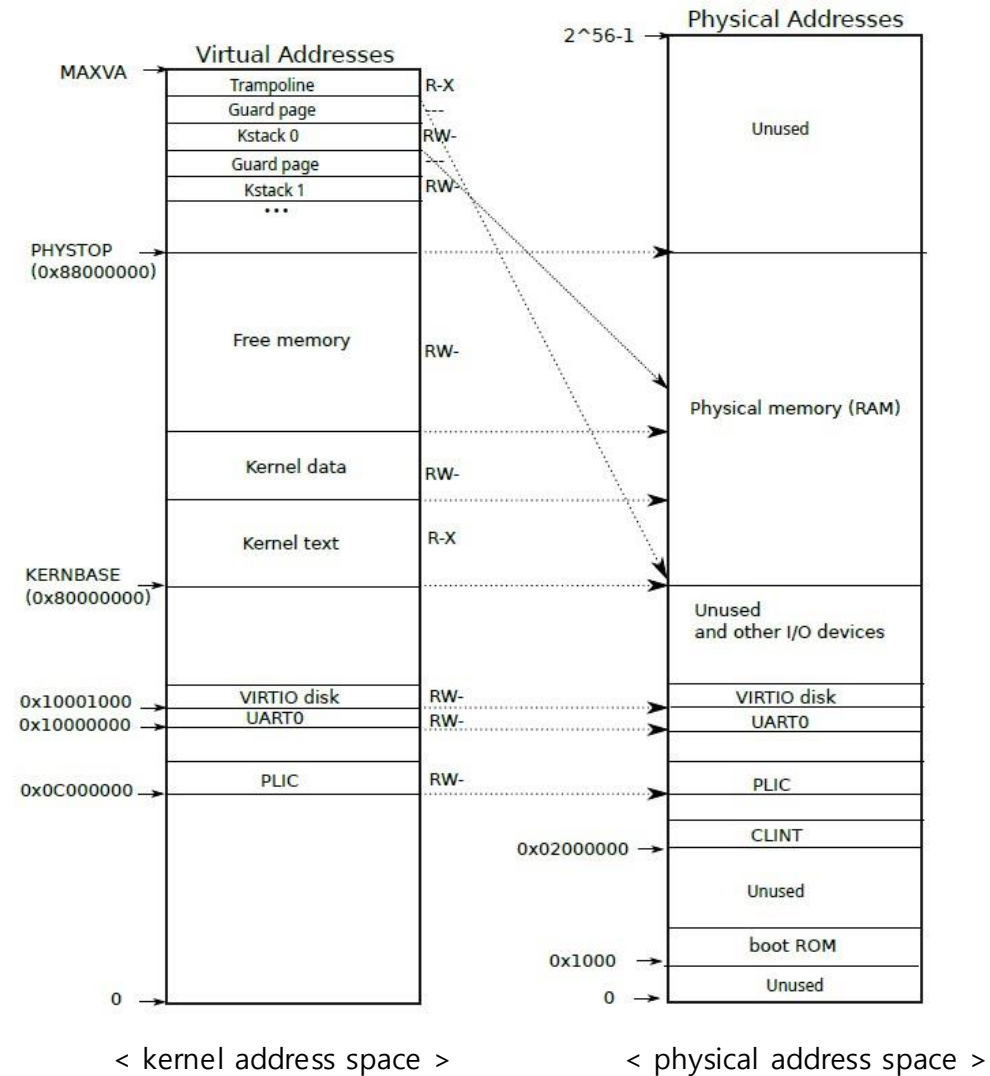
```
21  pagetable_t
22  kvmmake(void)
23  {
24      pagetable_t kpgtbl;
25
26      kpgtbl = (pagetable_t) kalloc();
27      memset(kpgtbl, 0, PGSIZE);
28
29      // uart registers
30      kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R ...
...
41      // map kernel data and the physical RAM we ...
42      kvmmap(kpgtbl, (uint64)etext, (uint64)etext ...
43
44      // map the trampoline for trap entry/exit to
45      // the highest virtual address in the kernel.
46      kvmmap(kpgtbl, TRAMPOLINE, (uint64)trampoline ...
47
48      // allocate and map a kernel stack for each ...
49      proc_mapstacks(kpgtbl);
50
51      return kpgtbl;
52  }
```

- `kvmmake()` initializes the kernel page table.
- It creates a page table that sets up a direct mapping between the kernel's virtual address space and corresponding physical addresses.
- The `kvmmap()` function uses `mappages()` to establish mappings in the kernel page table.

```
$ vim kernel/vm.c
```

```
57  void
58  kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa...
59  {
60      if(mappages(kpgtbl, va, sz, pa, perm) != 0)
61          panic("kvmmap");
62  }
```

Create and Initialize Kernel's Page Table



Create and Initialize Kernel's Page Table

```
$ vim kernel/vm.c
```

```
52 // Initialize the one kernel_pagetable
53 void
54 kvminit(void)
55 {
56     kernel_pagetable = kvmmake();
57 }
```

```
$ vim kernel/vm.c
```

```
61 void
62 kvminithart()
63 {
64     // wait for any previous writes to ...
65     sfence_vma();
66
67     w_satp(MAKE_SATP(kernel_pagetable));
68
69     // flush stale entries from the TLB.
70     sfence_vma();
71 }
```

- `kvminit()` initializes the kernel page table by creating direct mappings for kernel code, data, devices, and trampoline.
 - by calling `kvmmake()`
- `kvminithart()` activates the kernel page table on each hart by writing it to the `satp` register and flushing the TLB.
- Both are called in the main function of `xv6`.

How do we use these Page Tables?

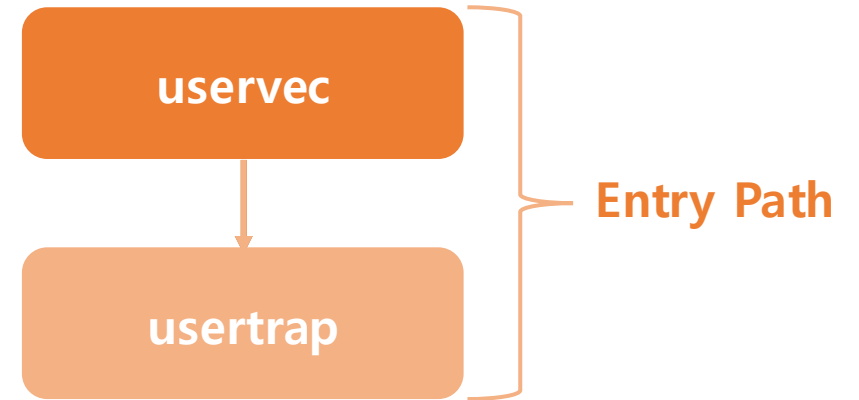


In mode switch !

Page Table Switching

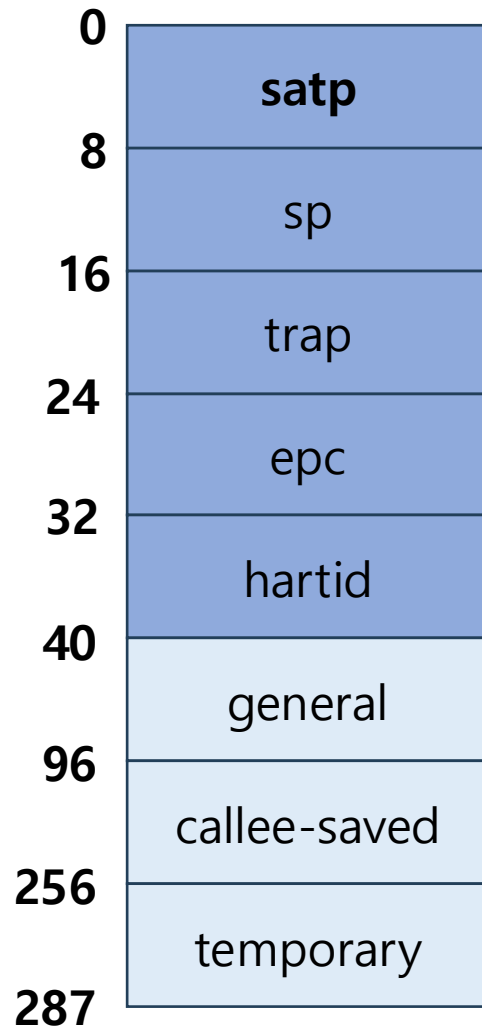
```
$ vim kernel/trampoline.S

22  uservec:
...
37      li a0, TRAPFRAME
...
84      # fetch the kernel page table address...
85      ld t1, 0(a0)
...
91      # install the kernel page table.
92      csrw satp, t1
...
```



- In **uservec**, the CPU loads the kernel page table address from the trapframe.
 - Therefore, during the handling process in the kernel, the kernel page table is fetched.

Recall: struct trapframe

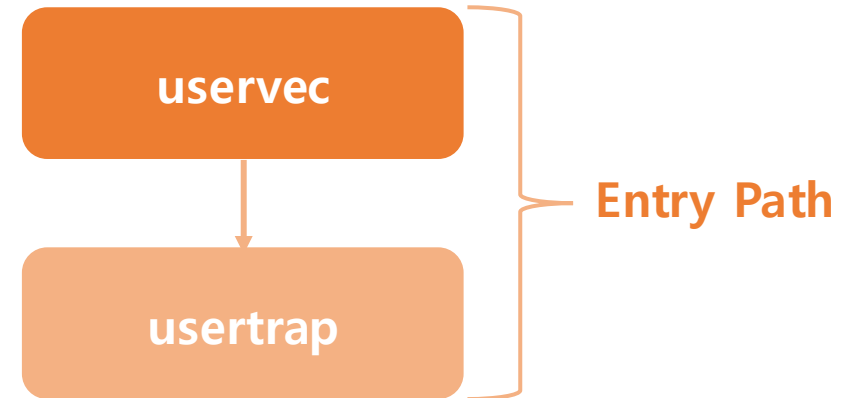


- `kernel_satp` : stores kernel page table
- `kernel_sp` : stores the top address of the kernel stack of the process
- `kernel_trap` : stores the address of the `usertrap()`
- `epc` : stores user program counter(pc)
- `kernel_hartid` : stores the hart id

Page Table Switching

```
$ vim kernel/trampoline.S

22  uservec:
...
37      li a0, TRAPFRAME
...
84      # fetch the kernel page table address...
85      ld t1, 0(a0)
...
91      # install the kernel page table.
92      csrw satp, t1
...
```

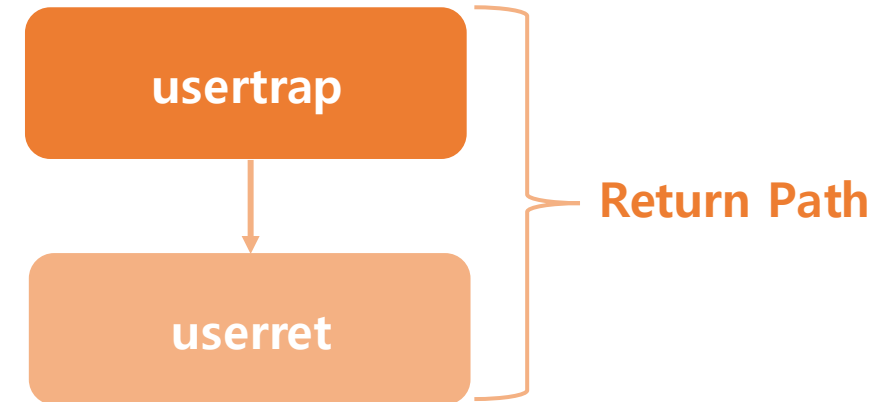


- In **uservec**, the CPU loads the kernel page table address from the trapframe.
 - Therefore, during the handling process in the kernel, the kernel page table is fetched.

Page Table Switching

```
$ vim kernel/trap.c

37  uint64
38  usertrap(void)
39  {
...
87    prepare_return();
...
90    uint64 satp = MAKE_SATP(p->pagetable);
...
93    return satp;
94  }
...
99  void
100  prepare_return(void)
101  {
...
115    p->trapframe->kernel_satp = r_satp();
...
131 }
```

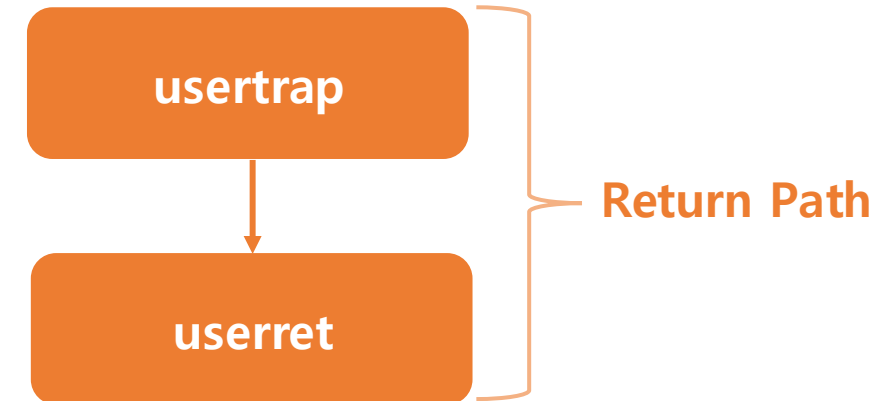


- In `prepare_return()`, CPU saves the **current SATP value (kernel page table)** into the trapframe.
 - So **uservec** can switch back to the kernel page table when a future trap occurs.
- After that, `usertrap()` returns the **user process's SATP value**, which will be used to switch back to user mode.

Page Table Switching

```
$ vim kernel/trampoline.S

22  uservec:
...
97   # call usertrap()
98   jalr t0
99
100  .globl userret
101  userret:
...
107  # switch to the user page table.
108  sfence.vma zero, zero
109  csrw satp, a0
110  sfence.vma zero, zero
...
```



- Upon returning from `usertrap()`, execution directly resumes at `userret` due to the `jalr` instruction.
- In `userret()`, the CPU loads the user page table address using the `satp` value returned in the `a0` register.

Thank you

